

LECTURE 4

SQL (Structured query language), basic SQL commands.

Objective: To provide students with systematic knowledge of the SQL language - its purpose, structure, and key commands for working with object-relational databases - and to develop skills in using SQL to create, modify, and manage databases, as well as to retrieve data from them.

Key Topics

1. Introduction to SQL
2. Structure and classification of SQL commands
3. Key DDL commands for creating and modifying database structure
4. Key DML commands for data manipulation; fundamentals of constructing SQL queries
5. Key SQL operators and functions
6. Ensuring data integrity using SQL

1. INTRODUCTION TO SQL

All database elements created via schema-management SQL statements are stored in a special set of tables known as the **data dictionary**. All this "data about the database" is referred to as **metadata**. You can query data dictionary tables using the SELECT statement, just as you would the tables you created yourself. In this way, the current data structures deployed in the database at runtime become accessible.

In your previous work with programming languages, you became accustomed to defining variables and data structures, using conditional logic (if-then-else) and looping constructs (do-while...end), and breaking code down into small, reusable components (objects, functions, procedures). The code is passed to a compiler, and the resulting executable performs exactly what you programmed it to do. Regardless of the language—whether Java, C#, C++, or any other procedural language—you have full control over the program's actions.

With SQL, however, you must relinquish the familiar control over execution, because SQL statements define the required input and output data, while the method of execution depends on a component of the database engine known as the **optimizer**.

SQL is not designed for writing complete applications on its own. If you need to create something more complex than a simple script for handling specific data, you will need to integrate SQL with a programming language.

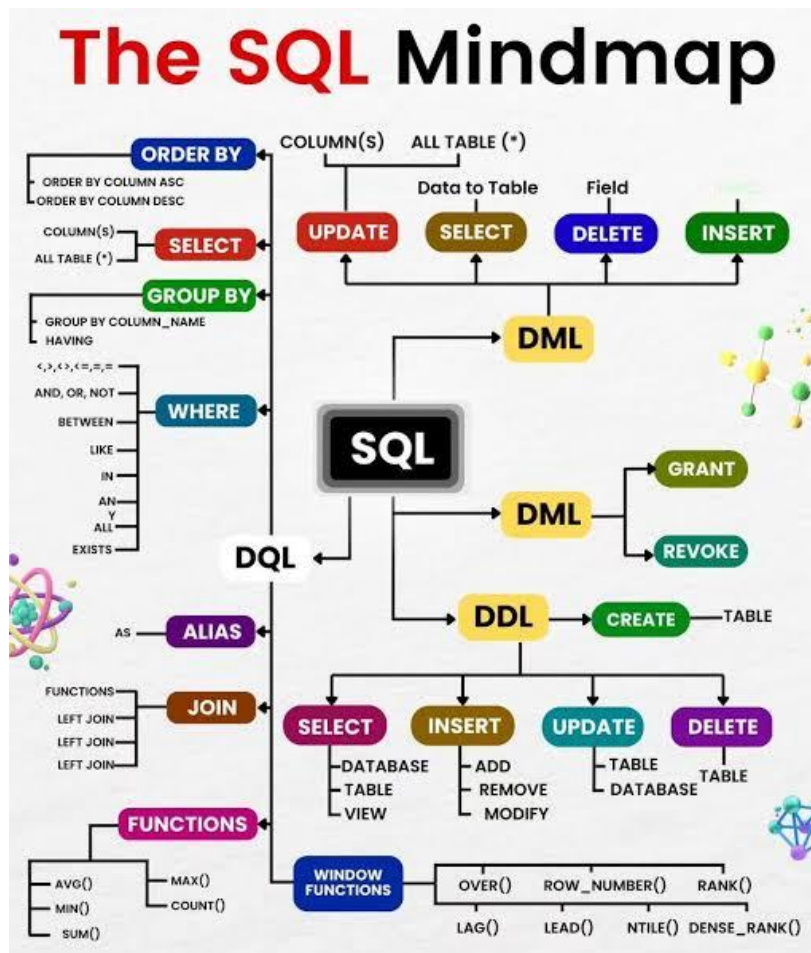
Relational databases have been on the market for over two decades. Some of the most popular products include:

- Oracle Database from Oracle Corporation
- SQL Server from Microsoft
- DB2 Universal Database from IBM
- Sybase Adaptive Server from Sybase

All these database servers perform essentially the same functions, although some are better equipped to handle very large or high-performance databases. Others excel at working with objects, very large files, XML documents, and so on. Furthermore, it is highly beneficial that all these servers are compatible with the latest ANSI SQL standard.

Alternatives to commercial database servers include the two most widely used open-source database servers: PostgreSQL and MySQL.

2. STRUCTURE AND CLASSIFICATION OF SQL COMMANDS



Main SQL command groups

SQL commands are divided into several categories based on their purpose:

- DDL (Data Definition Language) – commands for managing the database structure (creating and modifying tables).
- DML (Data Manipulation Language) – commands for working with the data itself (adding, modifying, deleting).
- DQL (Data Query Language) – reading and retrieving data.
- DCL (Data Control Language) – managing user access rights.

What is a table?

In a DBMS, data is stored in database objects called tables. A table is typically a collection of related data consisting of a specific number of columns and rows.

A table is the most common and simplest form of data storage in an RDBMS. Here is an example of a users table:

userId	userName	age	city	status
1	Igor	25	Moscow	active
2	Vika	26	Ekaterinburg	inactive
3	Elena	27	Ekaterinburg	active
4	Oleg	28	Moscow	inactive

What is a field?

Every table consists of small parts called fields. The fields in the ``users`` table are ``userId``, ``userName``, ``age``, ``city``, and ``status``. A field is a table column designed to store specific information about each record in the table.

Note: Instead of ``userId`` and ``userName``, one could use ``id`` and ``name``, respectively. However, when working with multiple objects that contain an ``id`` property, it can be difficult to determine which object the identifier belongs to — especially if, like me, you frequently use destructuring. As for the word ``name``, it often turns out to be reserved — meaning it is already in use within the environment where the code runs — so I try to avoid using it.

What is a record or row?

A record or row is any single entry within a table. The ``users`` table contains 5 records. Simply put, a record is a horizontal entry in the table.

What is a column?

A column is a vertical entry in a table that contains all the information related to a specific field. In the ``users`` table, one of the columns is ``city``, which contains the names of the cities where the users live.

What is a null value?

A null value (NULL) is a field value that is empty; in other words, it represents the absence of a value. It is important to understand that a null value differs from the number 0 and from a field containing spaces. A field with a null value is one that was left empty when the record was created. It should also be noted that in some database management systems (DBMS), an empty string (") is treated as NULL, whereas in others, they are considered distinct values.

Constraints

Constraints are rules applied to data. They are used to restrict the data that can be written to a table. This ensures the accuracy and integrity of the data in the database.

Constraints can be defined at either the column level or the table level.

Among the most common constraints are the following:

- NOT NULL — the column cannot contain a null value
- DEFAULT — the default value for the column
- UNIQUE — all values in the column must be unique
- PRIMARY KEY — the primary key, a unique identifier for a record in the current table
- FOREIGN KEY — a foreign key, a unique identifier for a record in another table (a table linked to the current one)
- CHECK — all values in the column must satisfy a specific condition
- INDEX — enables fast data writing and retrieval

Any constraint can be removed using the ``ALTER TABLE`` command combined with ``DROP CONSTRAINT`` and the constraint name. Some implementations provide shortcuts for removing constraints and the option to disable constraints instead of deleting them.

3. KEY DDL COMMANDS FOR CREATING AND MODIFYING DATABASE STRUCTURE

Key DDL commands

- CREATE — creates a new database object (the database itself, a table, an index, or a view).
- ALTER — modifies the structure of an existing object (e.g., adds or removes a column in a table).
- DROP — completely removes an object along with all the data contained within it.

- TRUNCATE — removes all rows from a table, clearing it while preserving its structure (often classified as DDL because it resets and recreates the underlying structure).
- RENAME — renames an existing object.

1. CREATE TABLE

Purpose: Creates a new table in the database.

Syntax:

```
CREATE TABLE <table_name1> (
    <col_name1><col_type1>,
    <col_name2><col_type2>,
    <col_name3><col_type3>
    PRIMARY KEY(<col_name1>),
    FOREIGN KEY(<col_name2>) REFERENCES <table_name2>(<col_name2>)
);
```

Integrity constraints when using CREATE TABLE

You may need to create constraints for specific columns in a table. When creating a table, you can specify the following constraints:

- a table cell cannot have a NULL value;
- primary key — PRIMARY KEY(col_name1, col_name2, ...);
- foreign key — FOREIGN KEY(col_name1, ..., col_namexn) REFERENCES table_name(col_name1, ..., col_namexn).

You can specify more than one primary key. In this case, the result is a composite primary key.

Example:

```
CREATE TABLE instructor (
    ID CHAR(5),
    name VARCHAR(20) NOT NULL,
    dept_name VARCHAR(20),
    salary NUMERIC(8,2),
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)
);
```

```
CREATE TABLE person (
    person_id SMALLINT UNSIGNED,
    fname VARCHAR(20),
    lname VARCHAR(20),
    gender CHAR(1),
    birth_date DATE,
    street VARCHAR(30),
    city VARCHAR(20),
    country VARCHAR(20),
    postal_code VARCHAR(20),
    CONSTRAINT pk_person PRIMARY KEY (person_id)
);
```

Everything in this expression should be clear, except for the last element. When defining a table, you must specify to the database server which column or columns will serve as the table's primary key. This is done by creating a constraint for the table. You can add a constraint of one of several types to the table definition. This specific constraint is a primary-key constraint; it is applied to the `person\_id` column and named `pk\_person`.

2. ALTER TABLE

Purpose: Modifies an existing table (e.g., adds a column).

Syntax:

```
ALTER TABLE name_table ADD name_column data_type;
```

Example:

```
ALTER TABLE cat      -- modify the 'cat' table
ADD age INTEGER;      -- add an "age" column of type INTEGER (integer)
```

id	name	age

3. DROP (TABLE, DATABASE, COLUMN, etc.)

Purpose: allows deleting objects in the database.

Syntax:

```
DROP object_type object_name;
;
```

Example:

```
DROP TABLE cat;
```

4. TRUNCATE TABLE (Clear Table)

Purpose: Removes all data from the table, but not the table itself.

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE Employees;
```

5. CREATE INDEX

Purpose: Creates an index on a table column for faster data access.

Syntax:

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

Example:

```
CREATE INDEX idx_surname ON Employees (Surname);
```

6. DROP INDEX

Purpose: Removes an index from the table.

Syntax:

```
DROP INDEX index_name ON table_name;
```

Note: Syntax may vary slightly depending on the database system.

Example:

```
DROP INDEX idx_surname ON Employees;
```

7. RENAME TABLE

Purpose: Renames an existing table.

Syntax:

```
ALTER TABLE table_name RENAME TO table_new_name;
```

Example:

```
ALTER TABLE Employees RENAME TO Staff;
```

Keep in mind that the exact syntax of these commands may vary slightly depending on the SQL database system (e.g., MySQL, PostgreSQL, SQL Server, etc.). Always consult the specific documentation for your database system for the most accurate information.

4. KEY DML COMMANDS FOR DATA MANAGEMENT

DML is one of its key components, as these commands allow us to work with data in tables that have already been created.

DML commands are tools for managing data within a database. They enable the addition, modification, deletion, and retrieval of data upon request, making databases flexible and useful for a wide range of tasks.

1. INSERT — Adding data

Using INSERT, we can add new rows (records) to tables.

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

For example, if we have a `Books` table containing information about books, we can use `INSERT` to add a new book to it.

Example:

```
INSERT INTO Books (Title, Author, YearPublished)
VALUES ('1984', 'George Orwell', 1949);
```

2. UPDATE — Updating Data

The UPDATE command allows you to modify existing data in a table.

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

For example, if we want to update a book's publication year, we can make this change using UPDATE.

Example:

```
UPDATE Books
SET YearPublished = 1950
WHERE Title = '1984';
```

This query updates the publication year of the book "1984" to 1950, but only for records where the Title is "1984".

3. DELETE — Deleting data

The DELETE command removes rows from a table.

```
DELETE FROM table_name
WHERE condition;
```

For example, if we want to delete a book from the Books table, we can use this command.

Example:

```
DELETE FROM Books
WHERE Title = '1984';
```

This query deletes the book titled "1984" from the “Books” table.

It is important to always include a `WHERE` clause; otherwise, all records in the table will be deleted.

4. SELECT — Data Retrieval

The `SELECT` command allows you to retrieve data from a table and display the desired records. It is the most frequently used DML command because it enables you to view and analyze data.

```
SELECT [DISTINCT] столбец1, столбец2, ...
FROM имя_таблицы
[WHERE условие]
[GROUP BY столбец_для_группировки]
[HAVING условие_для_группы]
[ORDER BY столбец [ASC | DESC]]
[LIMIT смещение, количество];
```

- SELECT — specifies the list of columns or expressions to be selected.
- The DISTINCT modifier removes duplicate rows.
- FROM — specifies the table or tables from which data is retrieved.
- WHERE — filters rows based on a specified condition before grouping.
- GROUP BY — groups rows with identical values.
- HAVING — filters the resulting groups after the GROUP BY operation.
- ORDER BY — sorts the result in ascending (ASC, default) or descending (DESC) order.
- LIMIT / OFFSET — limits the number of returned rows.

The database processes query components not in the order we write them, but according to the following sequence:

1. FROM (joins tables and retrieves source data)
2. WHERE (filters out unwanted rows)
3. GROUP BY (forms groups)
4. HAVING (filters out unwanted groups)
5. SELECT (calculates expressions and columns)
6. DISTINCT (removes duplicates)
7. ORDER BY (sorts the result)
8. LIMIT / OFFSET (extracts the required range)

Example:

```
SELECT Title, Author
FROM Books
```

WHERE YearPublished > 1950;

This query returns the title and author of all books published after 1950.

```
vijinimallawaarachchi — mysql -u root -p — 81x35

mysql> SELECT * FROM course WHERE dept_name='Comp. Sci.';
+-----+-----+-----+-----+
| course_id | title                                | dept_name | credits |
+-----+-----+-----+-----+
| CS-101    | Intro. to Computer Science          | Comp. Sci. | 4       |
| CS-190    | Game Design                        | Comp. Sci. | 4       |
| CS-315    | Robotics                          | Comp. Sci. | 3       |
| CS-319    | Image Processing                   | Comp. Sci. | 3       |
| CS-347    | Database System Concepts           | Comp. Sci. | 3       |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT * FROM course WHERE credits>3;
+-----+-----+-----+-----+
| course_id | title                                | dept_name | credits |
+-----+-----+-----+-----+
| BIO-101    | Intro. to Biology                  | Biology   | 4       |
| BIO-301    | Genetics                          | Biology   | 4       |
| CS-101     | Intro. to Computer Science         | Comp. Sci. | 4       |
| CS-190     | Game Design                        | Comp. Sci. | 4       |
| PHY-101    | Physical Principles                | Physics   | 4       |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT * FROM course WHERE dept_name='Comp. Sci.' AND credits>3;
+-----+-----+-----+-----+
| course_id | title                                | dept_name | credits |
+-----+-----+-----+-----+
| CS-101    | Intro. to Computer Science          | Comp. Sci. | 4       |
| CS-190    | Game Design                        | Comp. Sci. | 4       |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

GROUP BY

The GROUP BY operator is often used with aggregate functions—such as COUNT, MAX, MIN, SUM, and AVG—to group output values. The next step after basic commands is window functions.

```
SELECT <col_name1>, <col_name2>, ...
FROM <table_name>
GROUP BY <col_name>;
```

Example:

Let's list the number of courses for each faculty:

```
SELECT COUNT(course_id), dept_name
FROM course
GROUP BY dept_name;
```

```
vijinimallawaarachchi — mysql -u root -p — 81x16

(mysql> SELECT COUNT(course_id), dept_name FROM course GROUP BY dept_name;
+-----+-----+
| COUNT(course_id) | dept_name |
+-----+-----+
| 3                | Biology   |
| 5                | Comp. Sci. |
| 1                | Elec. Eng. |
| 1                | Finance   |
| 1                | History   |
| 1                | Music     |
| 1                | Physics   |
+-----+-----+
7 rows in set (0.00 sec)

mysql>
```


HAVING

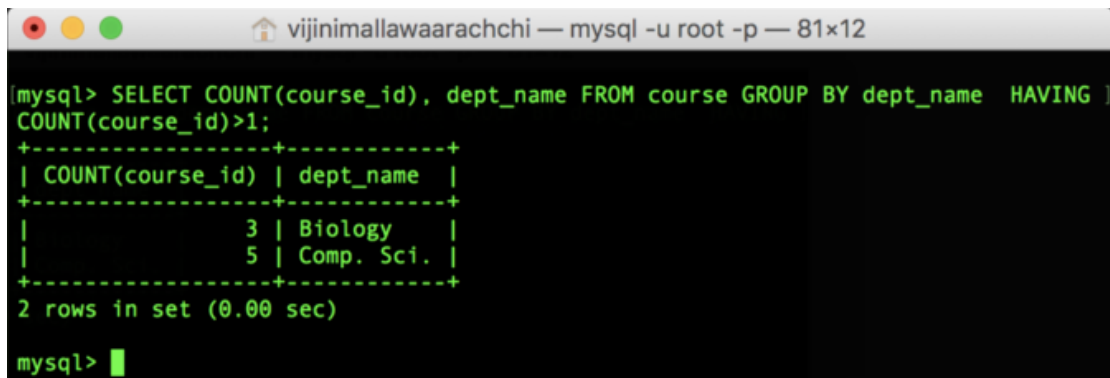
The HAVING keyword was added to SQL because WHERE cannot be used with aggregate functions.

```
SELECT <col_name1>, <col_name2>, ...  
FROM <table_name>  
GROUP BY <column_name>  
HAVING <condition>
```

Example:

Let's list the faculties that have more than one course:

```
SELECT COUNT(course_id), dept_name  
FROM course  
GROUP BY dept_name  
HAVING COUNT(course_id)>1;
```



The screenshot shows a MySQL terminal window with the title 'vijinimallawaarachchi — mysql -u root -p — 81x12'. The user has entered the following query: `SELECT COUNT(course_id), dept_name FROM course GROUP BY dept_name HAVING COUNT(course_id)>1;`. The output is displayed in a table format with two columns: 'COUNT(course_id)' and 'dept_name'. The results are: 3 for 'Biology' and 5 for 'Comp. Sci.'. The terminal also shows '2 rows in set (0.00 sec)' and the prompt 'mysql>'.

COUNT(course_id)	dept_name
3	Biology
5	Comp. Sci.

ORDER BY

ORDER BY is used to sort query results in descending or ascending order. ORDER BY sorts in ascending order if the sort method (ASC or DESC) is not specified.

```
SELECT <col_name1>, <col_name2>, ...  
FROM <table_name>  
ORDER BY <col_name1>, <col_name2>, ... ASC|DESC;
```

Example:

Let's generate a list of courses sorted by the number of credits in ascending and descending order:

```
SELECT * FROM course ORDER BY credits;  
SELECT * FROM course ORDER BY credits DESC;
```

BETWEEN

BETWEEN is used to select data values from a specific range. Numeric and text values, as well as dates, can be used.

```
SELECT <col_name1>, <col_name2>, ...  
FROM <table_name>  
WHERE <col_name> BETWEEN <value1> AND <value2>;
```

Example:

Let's list the instructors whose salary is greater than 50,000 but less than 100,000:

```
SELECT * FROM instructor  
WHERE salary BETWEEN 50000 AND 100000;
```

LIKE

The LIKE operator is used in a WHERE clause to specify a search pattern for matching values.

There are two wildcards used with LIKE:

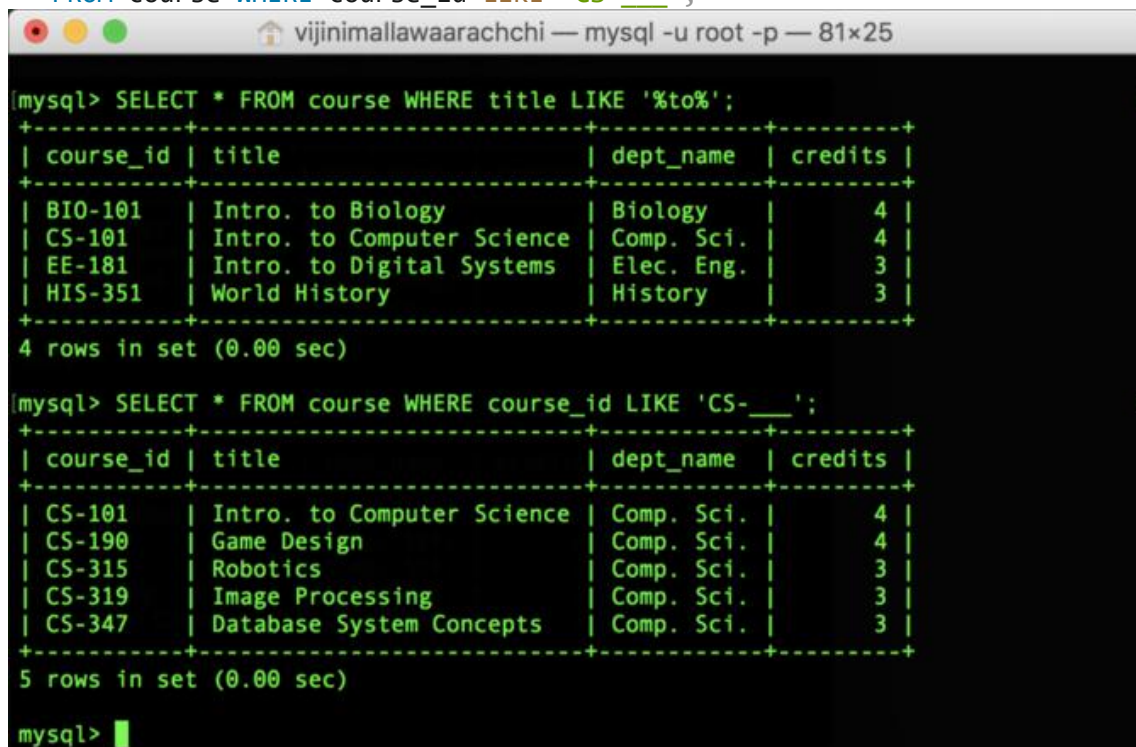
- % (zero, one, or multiple characters);
- _ (a single character).

```
SELECT <col_name1>, <col_name2>, ...  
FROM <table_name>  
WHERE <col_name> LIKE <pattern>;
```

Example:

Let's list the courses whose names contain "to" and the courses whose names start with "CS-":

```
SELECT * FROM course WHERE title LIKE '%to%';  
SELECT * FROM course WHERE course_id LIKE 'CS-__';
```



The screenshot shows a MySQL terminal window with the following content:

```
mysql> SELECT * FROM course WHERE title LIKE '%to%';  
+-----+-----+-----+-----+  
| course_id | title                | dept_name | credits |  
+-----+-----+-----+-----+  
| BIO-101   | Intro. to Biology    | Biology   | 4       |  
| CS-101    | Intro. to Computer Science | Comp. Sci. | 4       |  
| EE-181    | Intro. to Digital Systems | Elec. Eng. | 3       |  
| HIS-351   | World History        | History   | 3       |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> SELECT * FROM course WHERE course_id LIKE 'CS-__';  
+-----+-----+-----+-----+  
| course_id | title                | dept_name | credits |  
+-----+-----+-----+-----+  
| CS-101    | Intro. to Computer Science | Comp. Sci. | 4       |  
| CS-190    | Game Design          | Comp. Sci. | 4       |  
| CS-315    | Robotics              | Comp. Sci. | 3       |  
| CS-319    | Image Processing      | Comp. Sci. | 3       |  
| CS-347    | Database System Concepts | Comp. Sci. | 3       |  
+-----+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql>
```

IN

You can use IN to specify multiple values for the WHERE operator:

```
SELECT <col_name1>, <col_name2>, ...  
FROM <table_name>  
WHERE <col_name> IN (<value1>, <value2>, ...);
```

Example:

Let's list the students from the Comp. Sci., Physics, and Elec. Eng. tracks:

```
SELECT * FROM student  
WHERE dept_name IN ('Comp. Sci.', 'Physics', 'Elec. Eng.');
```

5. MERGE — Data Merging

The MERGE statement combines the capabilities of INSERT, UPDATE, and DELETE and is used to synchronize data between two tables. It allows you to add, update, or delete records in a target table based on data from a source table.

Example:

```
MERGE INTO Inventory AS inv
USING NewStock AS ns
ON inv.ProductID = ns.ProductID
WHEN MATCHED THEN
    UPDATE SET inv.Quantity = ns.Quantity
WHEN NOT MATCHED THEN
    INSERT (ProductID, ProductName, Quantity)
    VALUES (ns.ProductID, ns.ProductName, ns.Quantity);
```

The Inventory table stores current stock levels, while NewStock contains data on new arrivals.

Result: updates data for matching records, adds new ones from NewStock, and removes records missing from the source.

5. KEY SQL OPERATORS AND FUNCTIONS

Aggregate Functions

While not strictly core SQL commands, these are nonetheless important to know. Aggregate functions are used to obtain a summary result based on the data being analyzed.

- COUNT(col_name) — returns the number of rows;
- SUM(col_name) — returns the sum of values in the specified column;
- AVG(col_name) — returns the average value of the specified column;
- MIN(col_name) — returns the minimum value in the specified column;
- MAX(col_name) — returns the maximum value in the specified column.

There are also several built-in functions for working with strings:

- CONCAT — concatenates strings
- LENGTH — returns the number of characters in a string
- TRIM — removes whitespace from the beginning and end of a string
- SUBSTRING — extracts a substring from a string
- REPLACE — replaces a substring within a string
- LOWER — converts string characters to lowercase
- UPPER — converts string characters to uppercase, etc.

for numbers:

- ROUND — rounds a number
- TRUNCATE — truncates a fractional number to a specified number of decimal places
- CEILING — returns the smallest integer greater than or equal to the current value
- FLOOR — returns the largest integer less than or equal to the current value
- POWER — raises a number to a specified power
- SQRT — returns the square root of a number
- RAND — generates a random floating-point number in the range from 0 to 1

6. ENSURING DATA INTEGRITY USING SQL

Ensuring data integrity via SQL mechanisms guarantees the correctness, accuracy, and consistency of information within the database at the DBMS level.

Key concepts and integrity levels

- **Data integrity** - the adherence of data to the rules and constraints established within the database.
- **Declarative integrity** - rules defined directly during table creation using DDL (CREATE TABLE).
- **Procedural integrity** - rules implemented through application logic, stored procedures, and triggers.

Key SQL Constraints

- **PRIMARY KEY:** Uniquely identifies each row in a table and prohibits NULL values.
- **FOREIGN KEY:** Maintains referential integrity between tables by linking data in one table to another.
- **UNIQUE:** Ensures that all values in a column or group of columns are distinct.
- **NOT NULL:** Requires a field to contain a value and prohibits it from being empty (NULL).
- **CHECK:** Restricts the range or format of possible values in a field (e.g., age > 0).
- **DEFAULT:** Automatically inserts a specified value if none is provided during data entry.

Procedural tools

- **Triggers (TRIGGER):** Special programs that execute automatically when an attempt is made to add (INSERT), modify (UPDATE), or delete (DELETE) data, in order to check complex conditions.
- **Stored procedures and transactions (COMMIT / ROLLBACK):** Allow groups of commands to be executed as a single unit, rolling back all changes in the event of an error.

QUIZ QUESTIONS:

1. What is the purpose of SQL, and how does it differ from procedural programming languages regarding command execution control?
2. Into what main categories are SQL commands divided? What are the purposes and usage characteristics of DDL, DML, DQL, and DCL commands?
3. What are the differences between the CREATE, ALTER, DROP, and TRUNCATE commands? Provide examples of their use.
4. What integrity constraints can be defined when creating a table using the CREATE TABLE command? Explain the purpose of PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK.
5. What are the differences between the INSERT, UPDATE, and DELETE commands? What role does the WHERE clause play when executing UPDATE and DELETE commands?
6. What is the structure of an SQL SELECT query? Explain the purpose of the keywords FROM, WHERE, GROUP BY, HAVING, and ORDER BY.
7. What is the difference between the WHERE and HAVING clauses? In what situations is it appropriate to use each of them?
8. How are the BETWEEN, IN, and LIKE operators used for data filtering? Explain the purpose of the special characters % and _ in the LIKE operator.
9. For what purposes are the aggregate functions COUNT(), SUM(), AVG(), MIN(), and MAX() used? How are they used in conjunction with the GROUP BY clause?
10. What SQL mechanisms ensure data integrity in an object-relational database? What is the difference between declarative and procedural data integrity enforcement?